

Evidencias de avance – Sistema de control y gestión de viáticos

#4_Sebastián Valerdi Lopez

Objetivo general:

Desarrollar un sistema web para la gestión y control de viáticos en CAPCEE, que permita registrar, administrar y consultar la información de manera eficiente, mejorando la organización de los recursos y reduciendo los errores que se pudieran dar a causa del manejo manual de datos.

Actividades realizadas:

Como parte del cuarto avance del proyecto “Sistema de control y gestión de viáticos”, se realizaron las siguientes actividades:

1. Modificación del diagrama E-R y migraciones

Se realizó un pequeño ajuste de diseño en el diagrama E-R, manteniendo la misma lógica y relaciones entre las entidades. Posteriormente, se ejecutaron las **migraciones correspondientes** de todas las tablas en la base de datos “**viaticos**”, asegurando que reflejaran correctamente la estructura definida en el diagrama.

2. Creación de modelos

Se desarrollaron los modelos correspondientes en Laravel, lo que permite **trabajar con los datos como objetos** y define las relaciones lógicas entre las tablas. Esto facilita que el framework pueda unir automáticamente la información relacionada entre distintas entidades.

3. Configuración de controladores y rutas

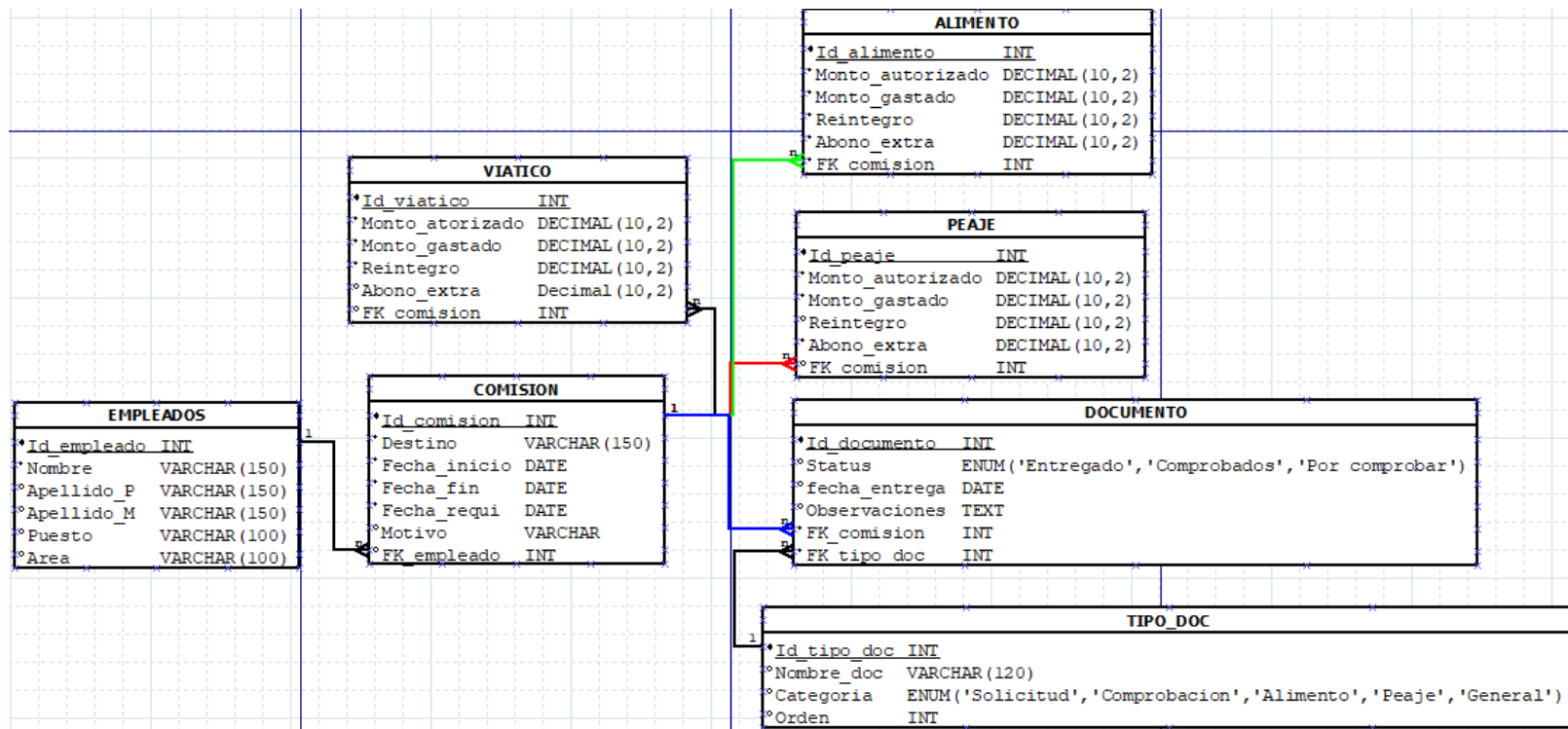
Se configuraron los **controladores**, que permiten manejar las funciones o acciones disponibles para cada tabla o entidad. En esta etapa, se implementó un **CRUD completo para la tabla “empleados”**, permitiendo registrar, consultar, actualizar y eliminar empleados. Además, se definieron las **rutas** que conectan la URL de la aplicación con los métodos correspondientes de los controladores.

4. Diseño de vistas y actualización del dashboard

Se elaboraron las **vistas correspondientes al CRUD de empleados**, incluyendo las pantallas de **index, create y edit**, permitiendo al usuario interactuar con la información de manera visual. Finalmente, se modificó el **dashboard** para integrar la nueva funcionalidad de agregar empleados al sistema, dejando preparado el entorno para futuras ampliaciones.

Se adjuntan imágenes con explicaciones que muestran los avances realizados:

1. Modificación del diagrama E-R y migraciones



Se movió viatico arriba de comision como se me indico en las observaciones realizadas.

Se hicieron las migraciones de las tablas del ER:

```
php artisan make:migration create_empleados_table
```

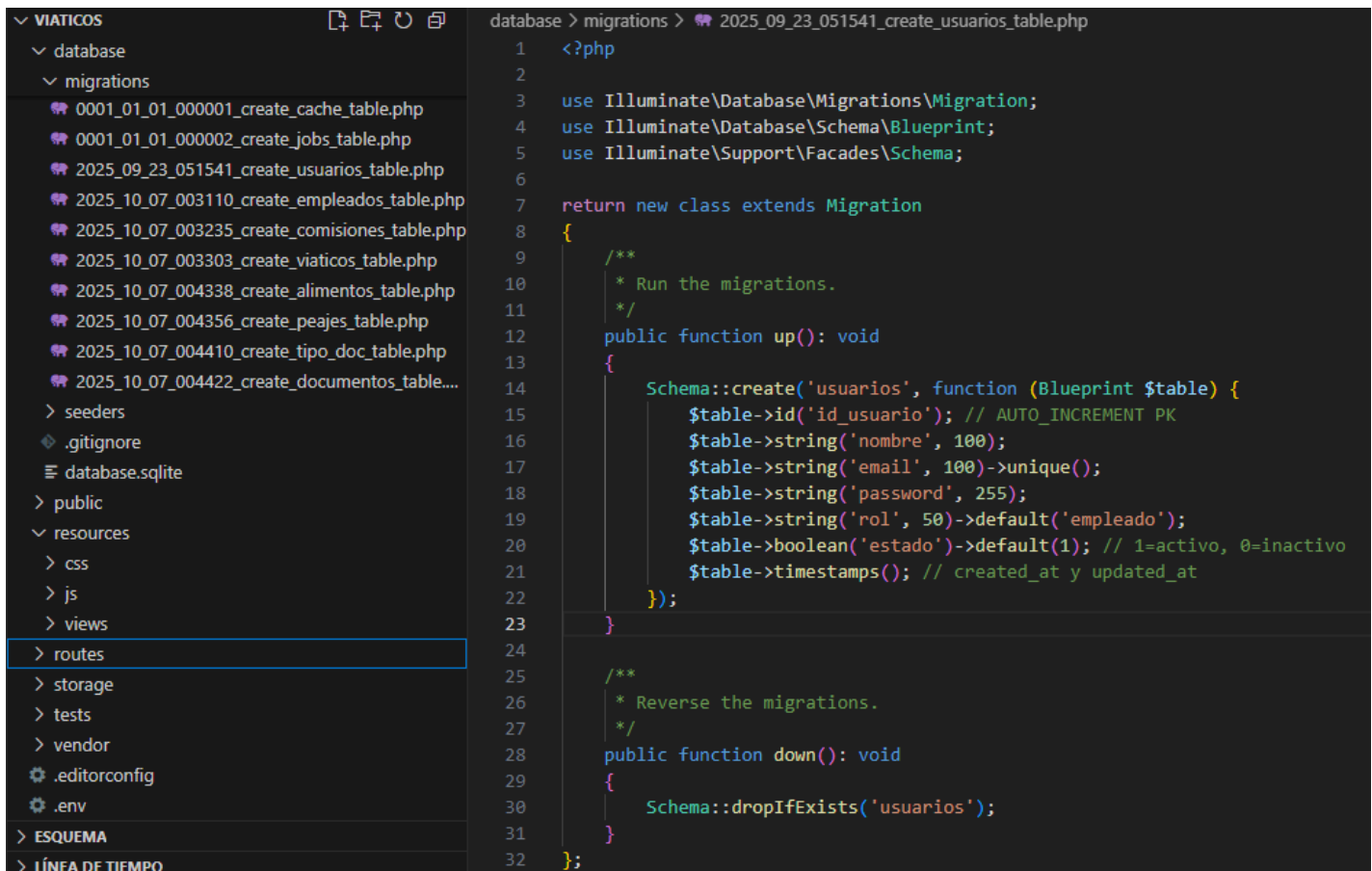
```
php artisan make:migration create_comisiones_table
```

```
php artisan make:migration create_viaticos_table
```

```
php artisan make:migration create_alimentos_table
```

```
php artisan make:migration create_peajes_table
```

```
php artisan make:migration create_tipo_doc_table  
php artisan make:migration create_documentos_table
```



```
database > migrations > 2025_09_23_051541_create_usuarios_table.php
1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\Schema;
6
7  return new class extends Migration
8  {
9      /**
10       * Run the migrations.
11       */
12     public function up(): void
13     {
14         Schema::create('usuarios', function (Blueprint $table) {
15             $table->id('id_usuario'); // AUTO_INCREMENT PK
16             $table->string('nombre', 100);
17             $table->string('email', 100)->unique();
18             $table->string('password', 255);
19             $table->string('rol', 50)->default('empleado');
20             $table->boolean('estado')->default(1); // 1=activo, 0=inactive
21             $table->timestamps(); // created_at y updated_at
22         });
23     }
24
25     /**
26      * Reverse the migrations.
27      */
28     public function down(): void
29     {
30         Schema::dropIfExists('usuarios');
31     }
32 };
```

Importa clases necesarias para crear la tabla:

- **Migration:** indica que este archivo es una migración.
- **Blueprint:** te permite definir cómo será tu tabla.
- **Schema:** sirve para crear, modificar o borrar tablas.

Esta clase tiene dos funciones principales: **up()** Crea o modifica tablas y **down()** Borra o revierte cambios.

Servidor: 127.0.0.1 » Base de datos: viaticos

Estructura

SQL

Buscar

Generar una consulta

Exportar

Importar

Operaciones

Privilegios

Más

Tabla	Acción	Filas	Tipo	Cotejamiento	Tamaño
☐ alimentos	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_unicode_ci	32.0 KB
☐ cache	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_unicode_ci	16.0 KB
☐ cache_locks	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_unicode_ci	16.0 KB
☐ comisiones	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_unicode_ci	32.0 KB
☐ documentos	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_unicode_ci	48.0 KB
☐ empleados	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	1	InnoDB	utf8mb4_unicode_ci	16.0 KB
☐ failed_jobs	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_unicode_ci	32.0 KB
☐ jobs	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_unicode_ci	32.0 KB
☐ job_batches	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_unicode_ci	16.0 KB
☐ migrations	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	11	InnoDB	utf8mb4_unicode_ci	16.0 KB
☐ password_reset_tokens	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_unicode_ci	16.0 KB
☐ peajes	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_unicode_ci	32.0 KB
☐ sessions	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	6	InnoDB	utf8mb4_unicode_ci	48.0 KB
☐ tipo_doc	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_unicode_ci	16.0 KB
☐ usuarios	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	2	InnoDB	utf8mb4_unicode_ci	32.0 KB
☐ viaticos	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_unicode_ci	32.0 KB

Tablas cargadas con éxito en MySQL.

2.Creación de modelos

VIATICOS
app
Http\ Controllers
Models
Alimento.php
Comision.php
Documento.php
Empleado.php
Peaje.php
TipoDoc.php
Usuario.php
Viatico.php
Providers
bootstrap
config
database
public
resources

app > Models > Empleado.php

```

1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Model;
6
7  class Empleado extends Model
8  {
9      protected $table = 'empleados'; // nombre de la tabla
10     protected $primaryKey = 'id_empleado'; // llave primaria
11
12     protected $fillable = [ // campos que pueden ser asignados masivamente
13         'nombre', 'apellido_p', 'apellido_m', 'puesto', 'area'
14     ];
15     // Relación: un empleado tiene muchas comisiones
16     public function comisiones()
17     {
18         return $this->hasMany(Comision::class, 'fk_empleado', 'id_empleado');
19     }
20 }

```

Se hicieron modelos de cada tabla.

-Modelo: Empleado.php

Función general:

Representa a los empleados del CAPCEE.

Cada empleado puede tener muchas comisiones (1:N).

-Modelo: Comision.php

Función general:

Guarda cada comisión o viaje de un empleado.

Está relacionada con **Empleado, Viático, Alimento, Peaje y Documento**.

-Modelo: Viatico.php

Función general:

Guarda los montos totales de viáticos asignados, gastados y sus diferencias (reintegro o abono extra).

Sirve como resumen general de una comisión.

-Modelo: Alimento.php

Función general:

Registra los gastos específicos de alimentos dentro de una comisión.

Aquí sí se aplican los límites del tabulador.

Modelo: Peaje.php

Función general:

Guarda los gastos por casetas o peajes en cada comisión.

A diferencia de alimentos, aquí el gasto es “libre” hasta un tope máximo (por ejemplo, \$1,000).

Modelo: TipoDoc.php

Función general:

Catálogo de tipos de documentos (por ejemplo: Oficio Interno, Requisición, Factura, XML, etc.).

Define a qué categoría pertenece cada tipo (Solicitud, Comprobación, etc.).

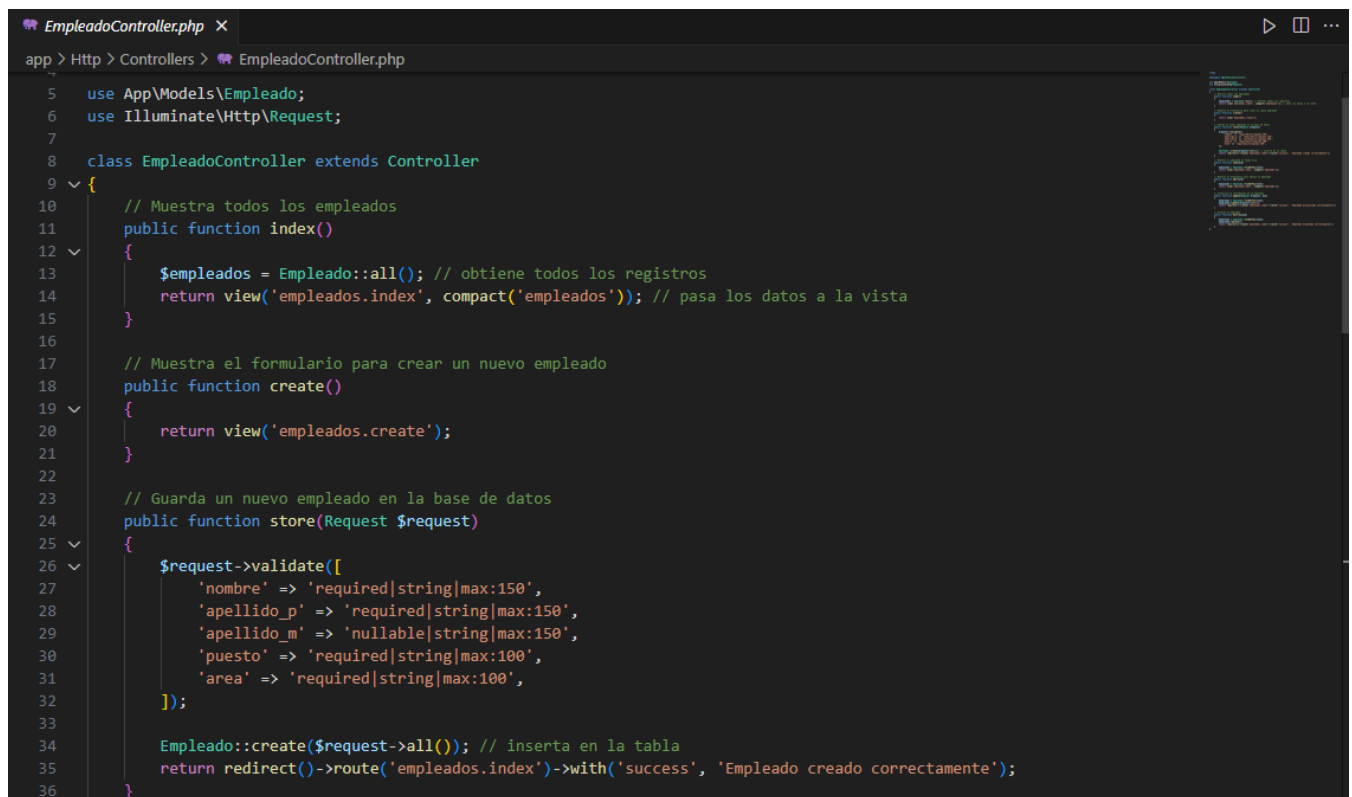
Modelo: Documento.php

Función general:

Guarda los documentos **subidos o registrados** por cada comisión (ya sea para solicitud o comprobación).

Cada documento tiene un tipo (fk_tipo_doc) y un estatus (Entregado, Por comprobar, etc.).

3. Configuración de controladores y rutas



```
EmpleadoController.php
app > Http > Controllers > EmpleadoController.php
5 use App\Models\Empleado;
6 use Illuminate\Http\Request;
7
8 class EmpleadoController extends Controller
9 {
10     // Muestra todos los empleados
11     public function index()
12     {
13         $empleados = Empleado::all(); // obtiene todos los registros
14         return view('empleados.index', compact('empleados')); // pasa los datos a la vista
15     }
16
17     // Muestra el formulario para crear un nuevo empleado
18     public function create()
19     {
20         return view('empleados.create');
21     }
22
23     // Guarda un nuevo empleado en la base de datos
24     public function store(Request $request)
25     {
26         $request->validate([
27             'nombre' => 'required|string|max:150',
28             'apellido_p' => 'required|string|max:150',
29             'apellido_m' => 'nullable|string|max:150',
30             'puesto' => 'required|string|max:100',
31             'area' => 'required|string|max:100',
32         ]);
33
34         Empleado::create($request->all()); // inserta en la tabla
35         return redirect()->route('empleados.index')->with('success', 'Empleado creado correctamente');
36     }
37 }
```

Código del controlador correspondiente a la tabla empleados con el cual hace un funcionamiento de CRUD.

```

9 {
37
38 // Muestra un empleado en específico
39 public function show($id)
40 {
41     $empleado = Empleado::findOrFail($id);
42     return view('empleados.show', compact('empleado'));
43 }
44
45 // Muestra el formulario para editar un empleado
46 public function edit($id)
47 {
48     $empleado = Empleado::findOrFail($id);
49     return view('empleados.edit', compact('empleado'));
50 }
51
52 // Actualiza la información de un empleado
53 public function update(Request $request, $id)
54 {
55     $empleado = Empleado::findOrFail($id);
56     $empleado->update($request->all());
57     return redirect()->route('empleados.index')->with('success', 'Empleado actualizado correctamente');
58 }
59
60 // Elimina un empleado
61 public function destroy($id)
62 {
63     $empleado = Empleado::findOrFail($id);
64     $empleado->delete();
65     return redirect()->route('empleados.index')->with('success', 'Empleado eliminado correctamente');
66 }

```

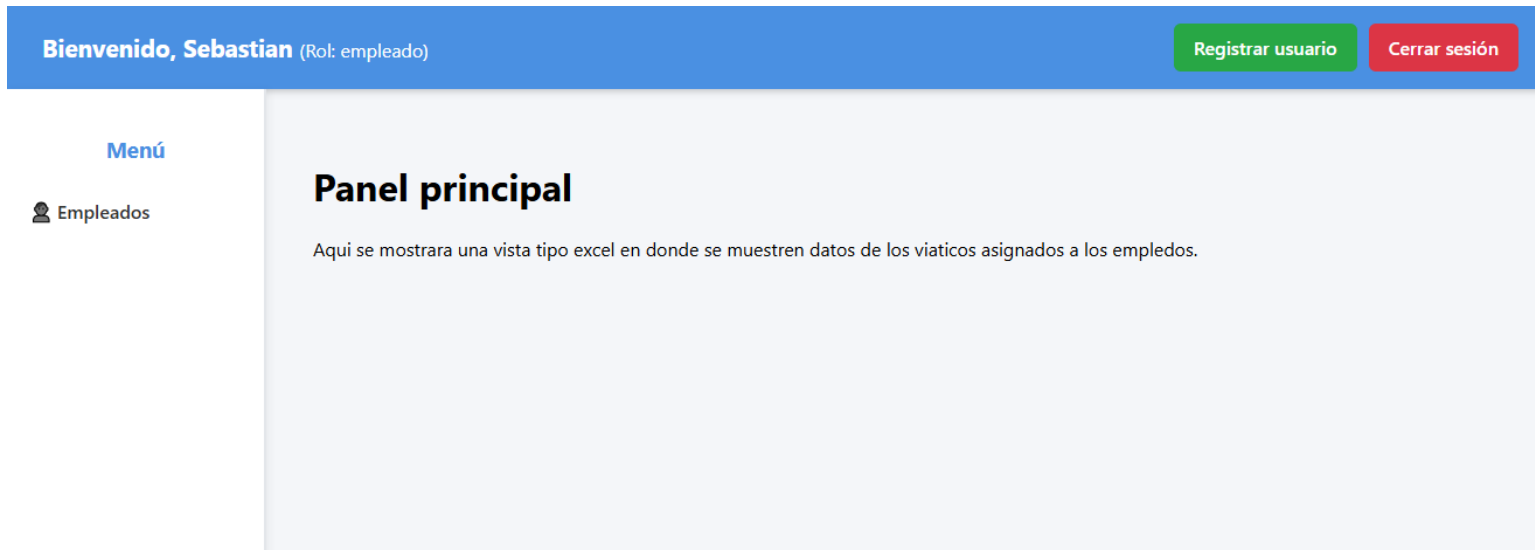
Ruta:

//Crea automáticamente la ruta CRUD (crear, leer, actualizar, eliminar) para empleados

Route::resource('empleados', EmpleadoController::class);

Ese Código se agraga en web.php

4. Diseño de vistas y actualización del dashboard



Se cambio el diseño del dashboard para acomodar mejor los botones que ya estaban y se le agrego un menú en donde estarán los apartados para navegar en este caso “empleados” y más adelante podría ser “Agregar un nuevo viatico”.



Este es el index de empleo aquí registre a alguien de prueba pero como se puede observar. Es un listo de los empleados registrados con la capacidad de editar y eliminar.

Registrar Empleado

Nombre:

Apellido Paterno:

Apellido Materno:

Puesto:

Área:

[Guardar](#)

[← Volver](#)

aquí podemos ver el formulario para registrar talmente funcional. Pues aquí mismo agregue al anterior empleado que mostré.

Editar Empleado

Nombre:

Apellido Paterno:

Apellido Materno:

Puesto:

Área:

[Actualizar](#)

[← Volver](#)

Este es el formulario para editar, vamos a editar el anterior empleado mostrado como ejemplo de que si funciona.

[← Regresar al Dashboard](#)

Listado de Empleados

[+ Nuevo Empleado](#)

ID	Nombre completo	Puesto	Área	Acciones
2	Arturo Perez Lopez	Auxiliar de contabilidad	Administrativa	Editar Eliminar

Como pueden observar se ha cambiado la información del empleado.

Por el momento seria todo, se esperan agregar mas cosas y funciones para el registro y cálculo de viáticos. Quedo al pendiente por cualquier observación o corrección, gracias.